ITI SOFTWARE TESTING TRACK

Automation of AutomationExercise.com Website

Graduation Project



Playwright (Java)



Selenium (Java)



Presented By : Omar Abdelaziz Fahmy

Date: December 2025

The Evolution of My Automation Project

Selenium → Playwright

Scope: Functional UI automation for the AutomationExercise demo e-commerce site



Table of Contents

1 Introduction to AutomationExercise

2 Automation Testing Overview

3 Manual Testing Limitations

4 Test Plan

5 Documentation & Test Coverage

6 Selenium Framework Architecture

7 Playwright Framework Architecture

8 Selenium Strengths & Challenges

9 Playwright Strengths & Challenges

10 References



Introduction to AutomationExercise

Overview:

- It is a practice website designed to mimic real-world e-commerce complexity.

Key Features:

- User Registration & Login
- Product Search & Filtering
- Add to Cart (Verify Prices)
- Checkout Process & Payment
- Download Invoice

Why this Website?

- It contains real-world challenges like
  - Heavy Ads
  - Pop-ups
  - Complex form handling
- There is a detailed documentations for the test cases



The Problem: Manual Testing Limitations

Time-Consuming & Inefficient

Manual regression testing demands significant time, delaying releases and increasing operational costs.

Human Error

Repetitive manual tasks inevitably lead to oversight and errors, compromising test coverage and accuracy.

Difficult to Scale

Expanding test coverage manually for a growing platform becomes impractical and unsustainable.

So why automate?

- Faster regression, repeatability, reliability.

Scope:

- Full coverage of automationexercise.com — registration, login, products, cart/checkout, contact us, invoices, API interactions, and more.

Test Plan

Purpose:
Define the testing approach and ensure full coverage for UI flow on AutomationExercise.com.

- **In-Scope:** Registration, Login, Products, Cart, Checkout, Orders, Contact Us.
- **Out-of-Scope:** Performance, Mobile, Security testing.
- **Testing Types:** Functional UI, Integration (UI + API), Regression.

Strategy:

- UI tests with POM
- Fluent Interface to enable method chaining.
- Data-driven using JSON.

Environment:

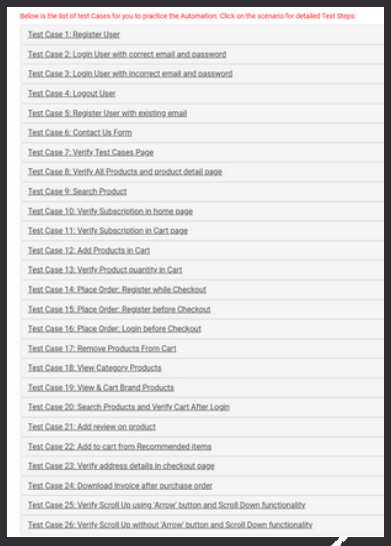
- Browsers: Chrome, Firefox, Edge (Selenium) & Chromium, Firefox, (Playwright).
- OS: Windows 11 & Linux.
- Reporting: Allure (steps, logs, screenshots).

Deliverables

- Automated test suites
- Allure reports
- logs
- logs files
- CI/CD Pipeline Integration
- Demo Video for the test execution
- Source Code on GitHub

AutomationExercise.com Documentation & Test Coverage

- **Functional areas covered:**
  - ✅ ○ **Registration**
  - ✅ ○ **Login**
  - ✅ ○ **Products search**
  - ✅ ○ **Product details**
  - ✅ ○ **Add to cart**
  - ✅ ○ **Checkout**
  - ✅ ○ **Orders/Invoices**
  - ✅ ○ **Contact Us**
  - ✅ ○ **API endpoints for user account setup**



Below is the list of test Cases for you to practice the Automation. Click on the scenario for detailed Test Steps:

Test Case 1: Register User
Test Case 2: Login User with correct email and password
Test Case 3: Login User with incorrect email and password
Test Case 4: Logout User
Test Case 5: Register User with existing email
Test Case 6: Contact Us Form
Test Case 7: Verify Test Cases Page
Test Case 8: Verify All Products and product detail page
Test Case 9: Search Product
Test Case 10: Verify Subscription in home page
Test Case 11: Verify Subscription in Cart page
Test Case 12: Add Products in Cart
Test Case 13: Verify Product quantity in Cart
Test Case 14: Place Order: Register while Checkout
Test Case 15: Place Order: Register before Checkout
Test Case 16: Place Order: Login before Checkout
Test Case 17: Remove Products From Cart
Test Case 18: View Category Products
Test Case 19: View & Cart Brand Products
Test Case 20: Search Products and Verify Cart After Login
Test Case 21: Add review on product
Test Case 22: Add to cart from Recommended items
Test Case 23: Verify address details in checkout page
Test Case 24: Download Invoice after purchase order
Test Case 25: Verify Scroll Up using 'Arrow' button and Scroll Down functionality
Test Case 26: Verify Scroll Up without 'Arrow' button and Scroll Down functionality

Selenium Framework Architecture

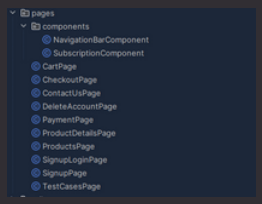
Tech Stack:

- Language: Java 21
- Build Tool: Maven
- Reporting: Allure
- Test Runner: TestNG
- API: RestAssured
- CI/CD: GitHub Actions
- Logging: Log4j2

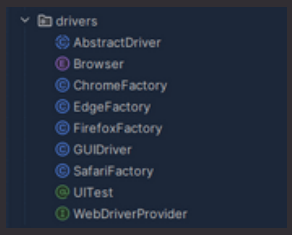


Key Features

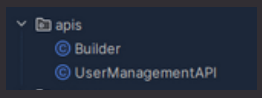
- **Page Object Model**



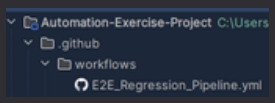
- **Singleton Driver & Cross-Browser support**



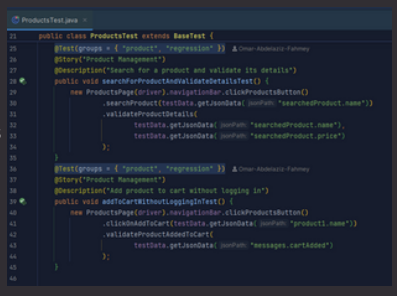
- **Hybrid Framework to also support API testing**



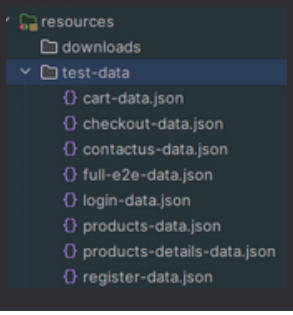
- **CI/CD using GitHub Actions**



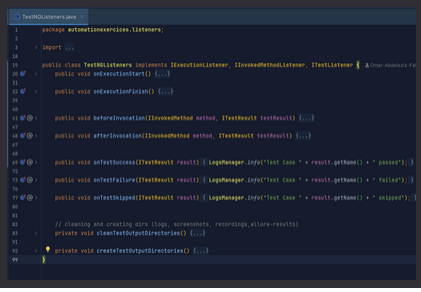
- **Fluent Interface to Enable method chaining**



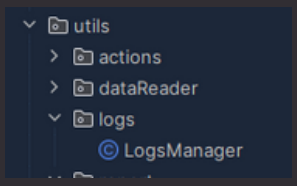
- **Data-Driven Testing**



- **Custom listeners**

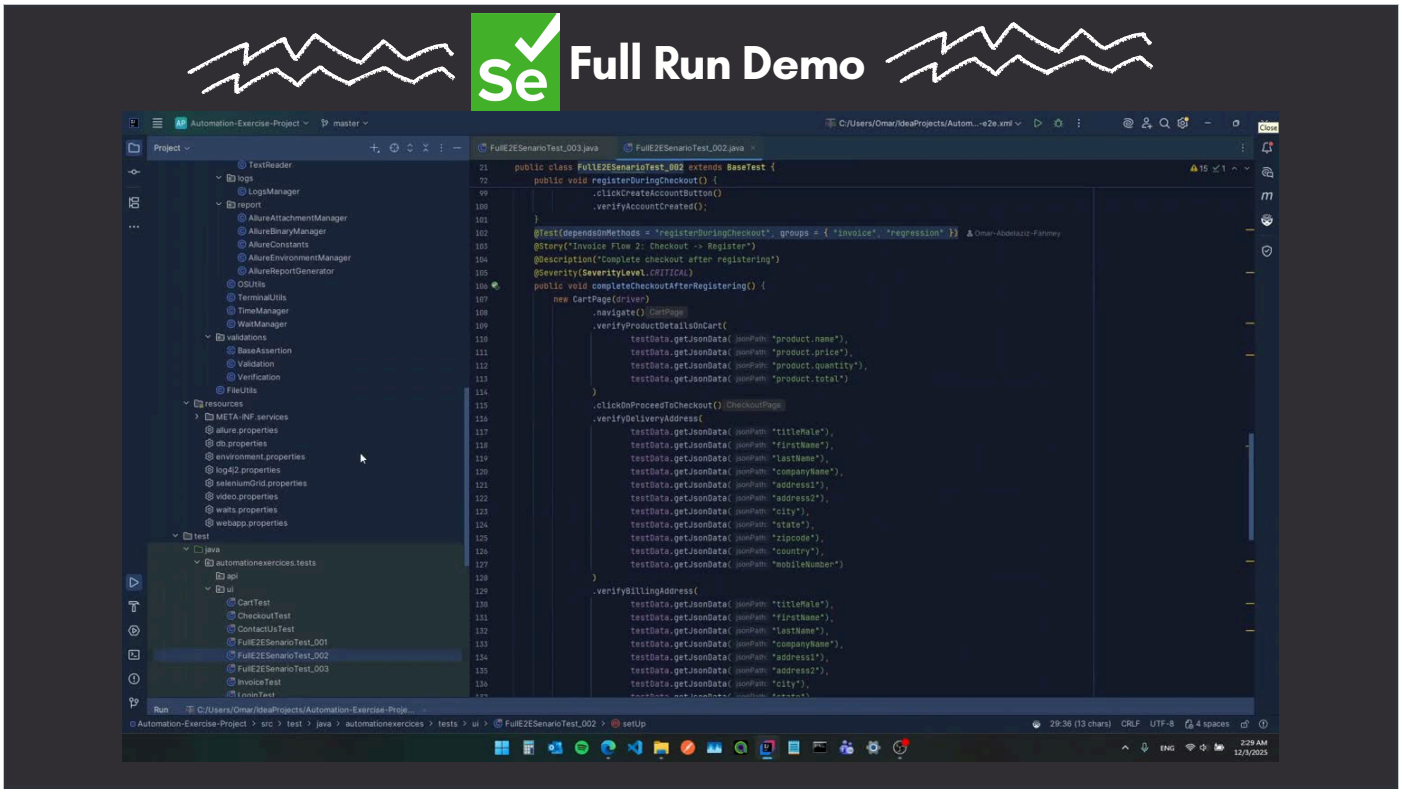


- **Log4j2 logging for real-time tracking**

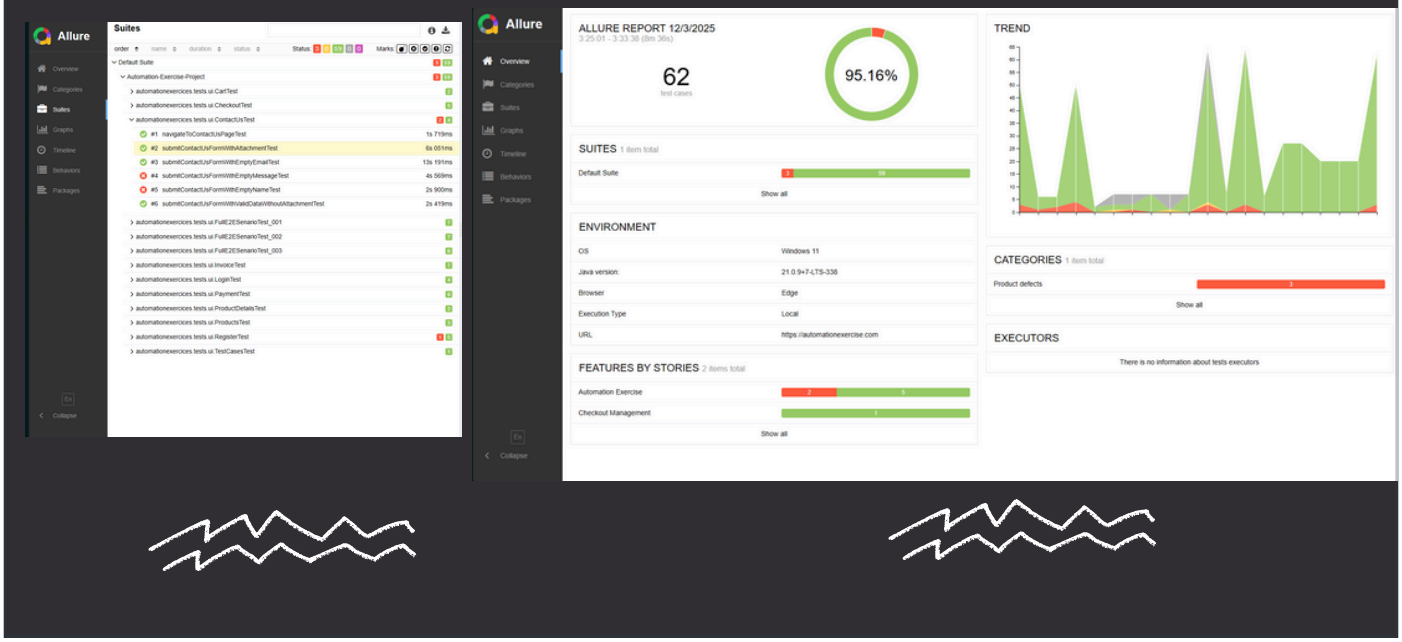


📂 Project Structure

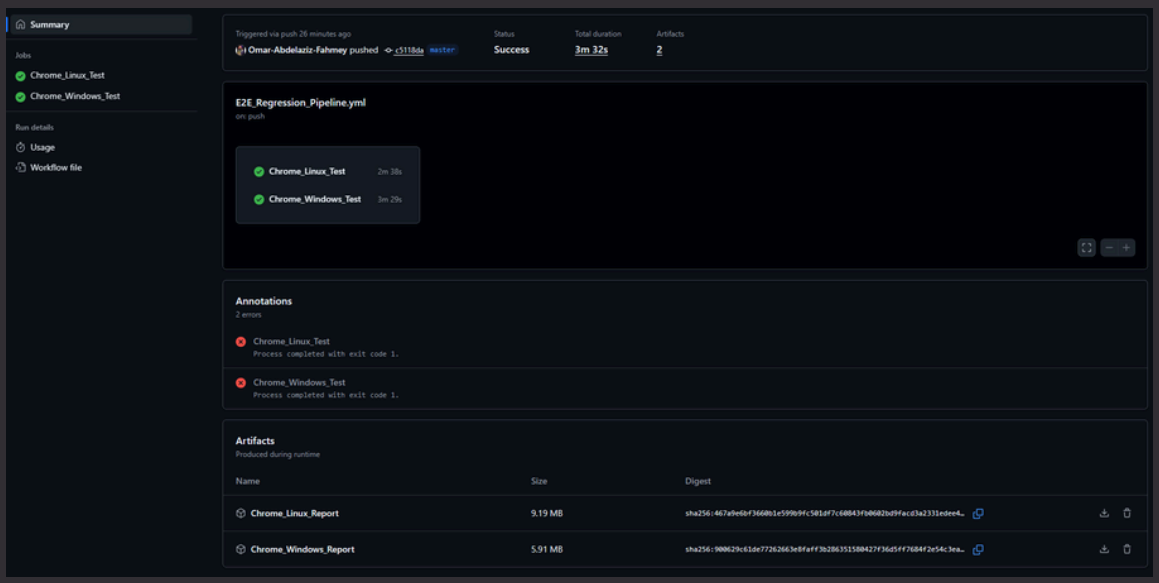
```
Automation-Exercise-Project
├── .github/                # GitHub Actions workflows (if applicable)
├── src/
│   ├── main/java/          # Source code
│   │   └── automationexercices/
│   │       ├── apis/       # API implementation classes
│   │       ├── drivers/    # WebDriver initialization and management
│   │       ├── listeners/  # TestNG listeners for reporting and logs
│   │       ├── pages/      # Page Object classes
│   │       ├── utils/      # Utility classes (JSON reader, Actions, etc.)
│   │       └── validations/# Assertion and Verification classes
│   └── test/java/          # Test code
│       └── automationexercices/
│           └── tests/
│               ├── api/    # API Test classes
│               └── ui/     # UI Test classes
├── src/test/resources/     # Test resources
│   ├── testData/           # JSON data files
│   └── properties/         # Configuration properties
├── pom.xml                 # Maven dependencies and build configuration
└── README.md               # Project documentation
```


Full Run Demo

Allure Results Summary



CI/CD Integration using GitHub Actions



Playwright Framework Architecture

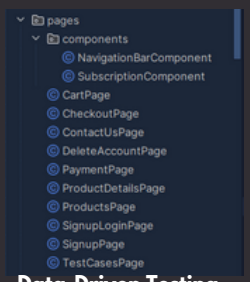
Tech Stack:

- Language: Java 21
- Build Tool: Maven
- Reporting: Allure
- Test Runner: TestNG
- API: RestAssured
- CI/CD: GitHub Actions
- Logging: Log4j2

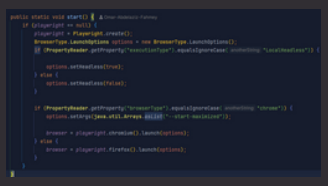


Key Features

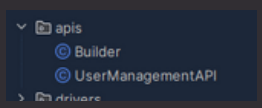
- **Page Object Model**



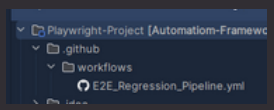
- **Cross-Browser support**



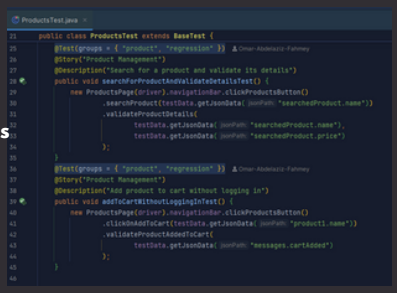
- **Hybrid Framework to also support API testing**



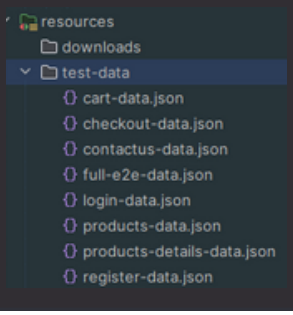
- **CI/CD using GitHub Actions**



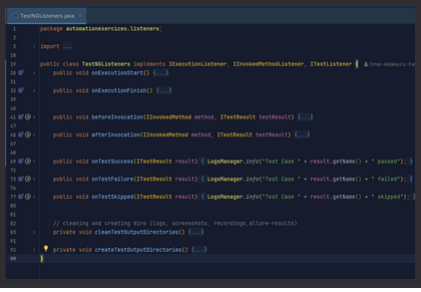
- **Fluent Interface to Enable method chaining**



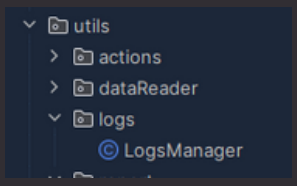
- **Data-Driven Testing**



- **Custom listeners**

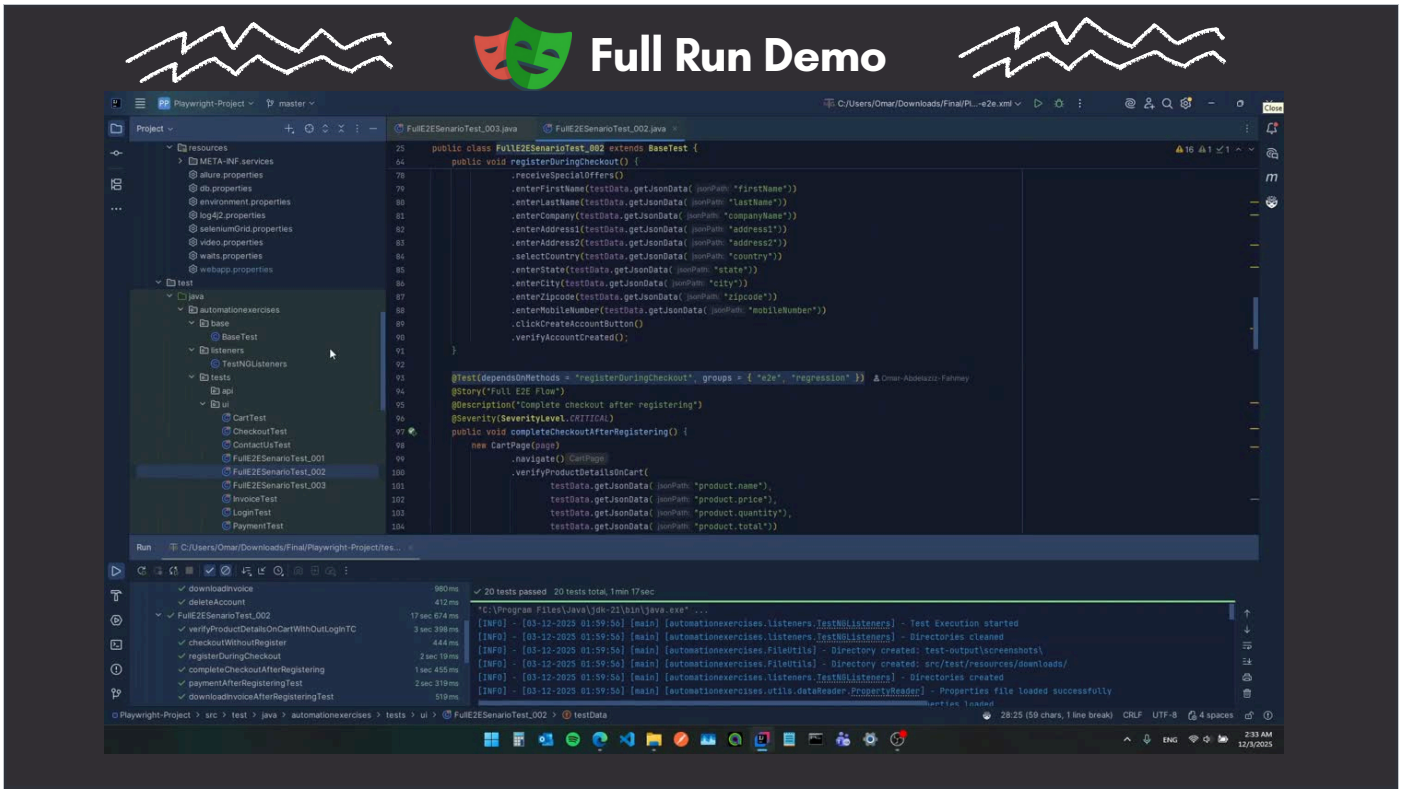


- **Log4j2 logging for real-time tracking**

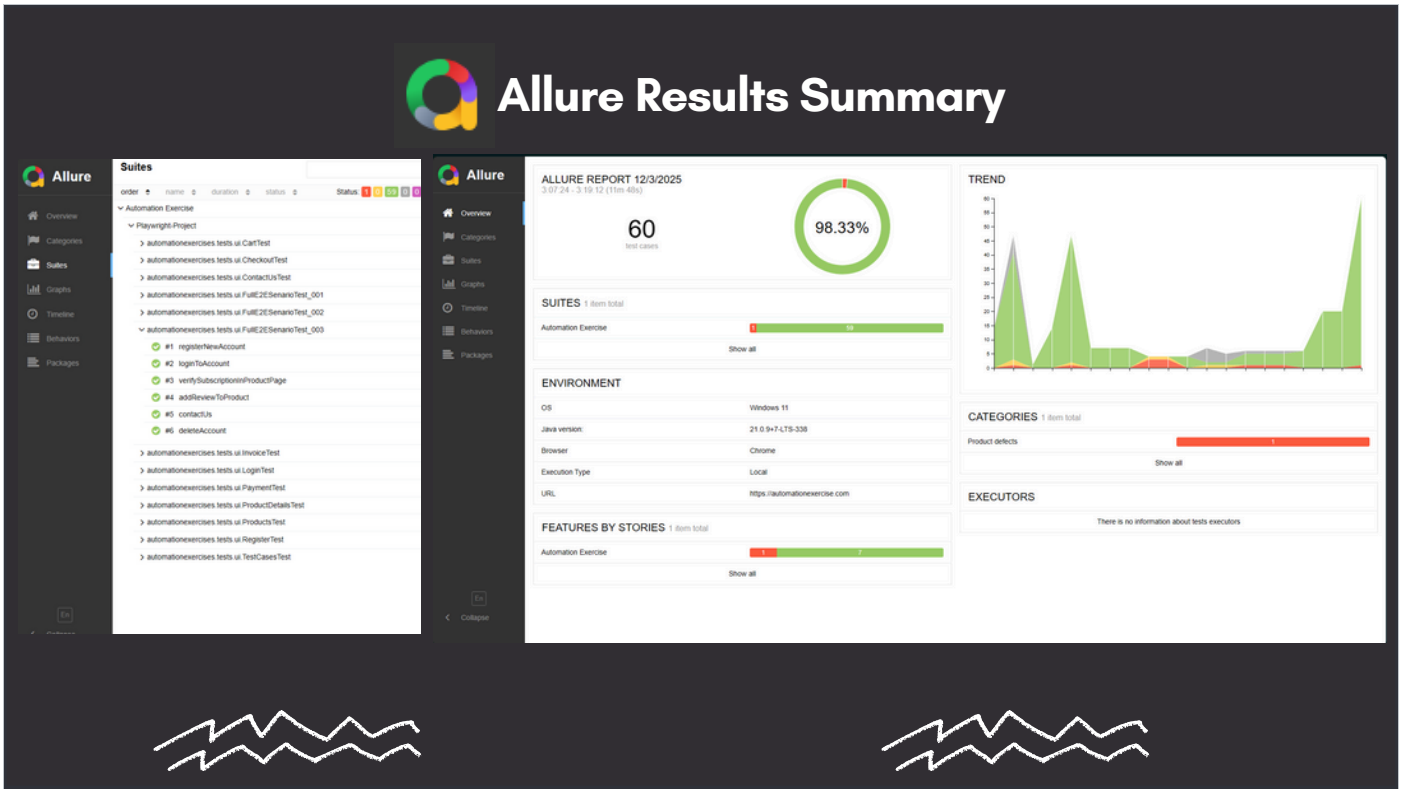


📂 Project Structure

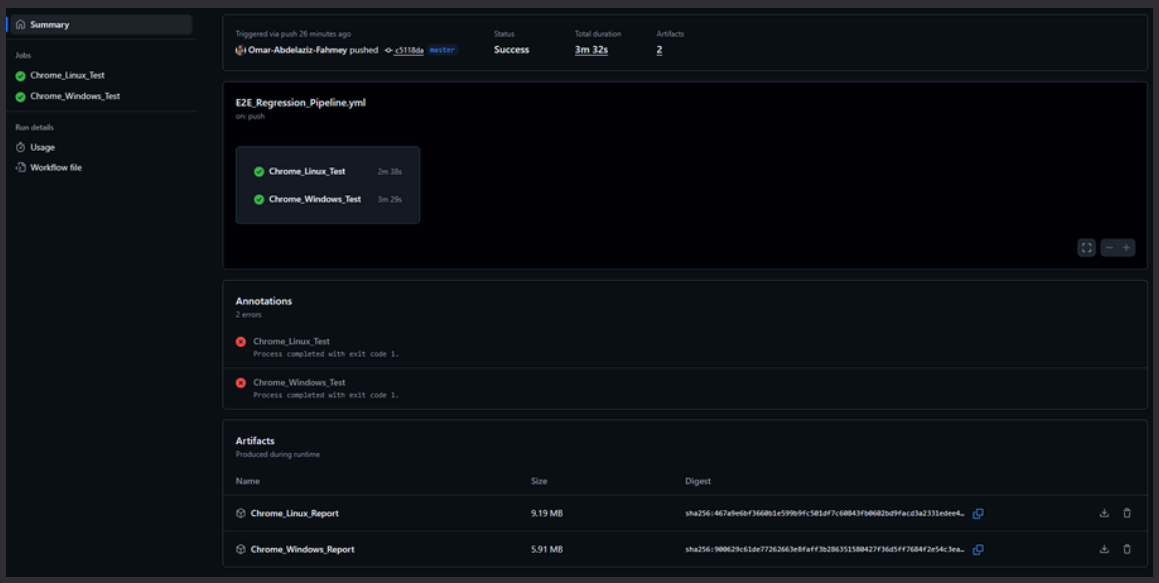
```
Playwright-Project
├── src/
│   ├── main/java/          # Source code
│   │   └── automationexercises/
│   │       ├── apis/       # API implementation classes
│   │       ├── pages/      # Page Object classes
│   │       ├── utils/      # Utility classes (JSON reader, etc.)
│   │       ├── FileUtils.java
│   │       └── PlaywrightManager.java
│   └── test/java/          # Test code
│       └── automationexercises/
│           ├── base/       # Base Test class
│           ├── listeners/  # TestNG listeners
│           └── tests/
│               └── ui/     # UI Test classes
├── src/test/resources/     # Test resources
│   ├── downloads/          # Downloaded files during tests
│   └── test-data/          # JSON data files
├── *.xml                   # TestNG Suite files (testng.xml, regression.xml, etc.)
├── pom.xml                 # Maven dependencies and build configuration
└── README.md               # Project documentation
```


Full Run Demo

Allure Results Summary



CI/CD Integration using GitHub Actions



Selenium Strengths

Industry Standard & Maturity

- The pioneer and most established browser automation tool.
- Over 15 years of development, bugs, and solutions.

Unmatched Language Support

- Official bindings for Java, Python, C#, JavaScript, Ruby.



Mature Ecosystem

- Largest community support (forums, Stack Overflow, blogs).
- Countless integrations, plugins, and frameworks.

Wide Browser Support

- Supports all major browsers (Chrome, Firefox, Safari, Edge, Internet Explorer)

Selenium Challenges

Flakiness & Need for Explicit Waits

- Requires manual handling of waits (implicit/explicit).
- A leading cause of unreliable ("flaky") tests.

Less built-in modern features

- limited native support for network mocking, tracing, video, and auto-waiting.



Slower Execution

- No direct access to browser's internal debugging protocol.

Complex Setup

- Requires separate driver executables for each browser.
- browser drivers, version compatibility.

Playwright Strengths

Modern Architecture & Speed

- Communicates directly via the browser's DevTools Protocol (CDP).
- Faster execution and more reliable control.

Auto-Waiting & Reduced Flakiness

- Built-in auto-wait for element reduces the need for manual timing.



Powerful Features Out-of-the-Box

- Excellent Trace Viewer, Code Generator, and Inspector.
- Visual debugging with UI Mode
- Network interception, request mocking, tracing, screenshots, video

Cross-engine support

- Chromium, Firefox, WebKit (good for Safari testing).

Playwright Challenges

Limited Language Support

- JavaScript/TypeScript, Python, C#, Java

Smaller Community

- Newer tool, fewer resources and tutorials available



No Legacy Support

- Does not support Internet Explorer or very old browser versions

Enterprise integration

- some CI / tooling workflows need adjustment when migrating from Selenium.

REFERENCES

- Selenium Documentation
- Playwright Documentation
- Allure Report Documentation
- Mastering Selenium with Java :أوتوميشن من الصفر للاحتراف
- TestNG Documentation

24


Thank You
REPOSITORIES
Selenium Version
Playwright Version
My Github Account
My LinkedIn Account